Report-HW4

Randomized Quicksort works by choosing a random pivot is selected using **random.randint(low, high)**. The partitioning process then moves all elements smaller than or equal to the pivot to the left side using comparisons and swaps inside a loop. After partitioning, the algorithm recursively calls itself on the left and right subarrays using **randomized_quicksort(arr, low, pivot_index-1)** and **randomized_quicksort(arr, pivot_index + 1, high)**.

```python
def randomized_quicksort(arr, low=None, high=None):

    if low is None:
        low = 0
    if high is None:
        high = len(arr) - 1
    if low < high:

        pivot_index = randomized_partition(arr, low, high)
        randomized_quicksort(arr, low, pivot_index - 1)
        randomized_quicksort(arr, pivot_index + 1, high)
    return arr
```

Heapsort works by building a max heap and repeatedly moving the largest element to the end, then fixing the heap until the array is sorted. the array is first converted into a max heap using a loop with **heapify(arr, n, i).** The heapify function ensures that the largest element is always at the root by comparing parent and child nodes. Then, the largest element is repeatedly swapped with the last element using **arr[0], arr[i] = arr[i], arr[0]**, and the heap is restored by calling **heapify(arr, i, 0)** until the array is fully sorted and the heap is restored by calling **heapify(arr, i, 0)** until the array is fully sorted. Inside **heapify**, the left and right children are calculated using *2 * root_index + 1 and 2 * root_index + 2*, and if a child is larger than the root, a swap is performed. The function then recursively calls itself on the affected subtree to maintain the max heap property.

```python
def heapify(arr, heap_size, root_index):

    largest = root_index

    left_child = 2 * root_index + 1
    right_child = 2 * root_index + 2
    if left_child < heap_size and arr[left_child] > arr[largest]:
        largest = left_child
    if right_child < heap_size and arr[right_child] > arr[largest]:
        largest = right_child
    if largest != root_index:
        arr[root_index], arr[largest] = arr[largest], arr[root_index]

        heapify(arr, heap_size, largest)

def heapsort(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)
    return arr
```

Best, worst, and average-case time complexity

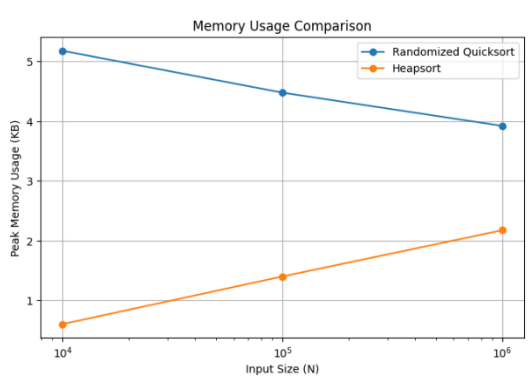
Randomized Quicksort has a best and average-case time complexity of **O(n log n)**, since the array is divided into roughly equal parts during partitioning. However, in the worst case, it can take **O(n^2)** time if the pivot repeatedly divides the array poorly, although using a random pivot makes this unlikely in practice. Heapsort has a time complexity of **O(n log n)** in the best, average, and worst cases because it always builds a heap and performs heap operations consistently. From the experimental results, both algorithms showed increasing running time as the input size increased; for example, Quicksort increased from about 0.23 seconds at 10^4 to about 31.79 seconds at 10^6.

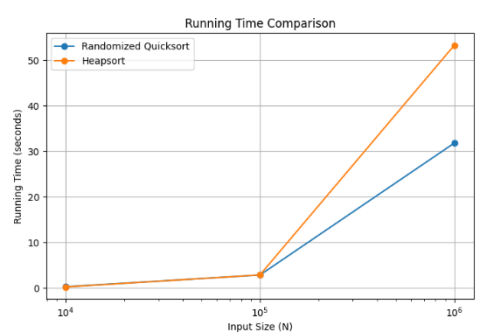
Comparison of Measured Performance with Theoretical Complexity

The measured performance matches the theoretical time complexities of both algorithms. As the input size increased, the running time also increased significantly, which is consistent with O (n log n) behavior. For example, Randomized Quicksort increased from about 0.23 seconds at $10^4$to about 31.79 seconds at 10^6, showing expected scaling. This confirms that both algorithms behave according to their theoretical analysis.

Performance Comparison (Runtime and Memory)

From the results, Randomized Quicksort performed faster than Heapsort as the input size increased, especially for larger inputs like 10^6. However, Heapsort consistently used less memory compared to Quicksort across all input sizes. This shows a trade-off between the two algorithms, where Quicksort is more efficient in terms of running time, while Heapsort is more memory efficient. Overall, both algorithms performed correctly, but their performance differs depending on whether speed or memory is prioritized.





The Memory Usage Graph shows that Heapsort uses less memory compared to Randomized Quicksort. Quicksort's memory usage slightly decreases, while Heapsort's increases with input size.

The running Time Graph shows that the running time of both algorithms increases as the input size grows. Randomized Quicksort performs faster than Heapsort for larger inputs, especially at 10^6.

Summary table

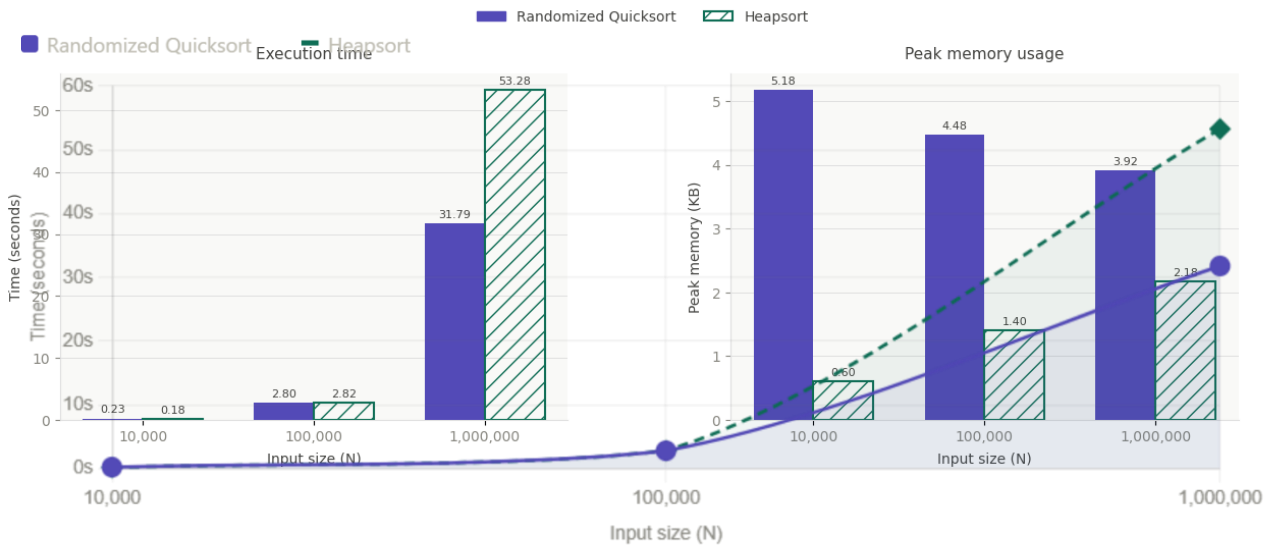
Benchmark Results — Randomized Quicksort vs Heapsort

N	Algorithm	Time (s)	Peak Memory (KB)	Status
10,000	Randomized Quicksort	0.22810	5.1787	Success
10,000	Heapsort	0.17720	0.6045	Success
100,000	Randomized Quicksort	2.80317	4.4775	Success
100,000	Heapsort	2.81849	1.4023	Success
1,000,000	Randomized Quicksort	31.79220	3.9219	Success
1,000,000	Heapsort	53.28350	2.1758	Success

Execution time — both algorithms perform nearly identically at smaller input sizes. The gap becomes visible at 1,000,000 elements, where Randomized Quicksort completes in ~31.8 seconds compared to Heapsort's ~53.3 seconds. This reflects Quicksort's better cache locality in practice, giving it a real-world speed advantage even though both share the same O(n log n) theoretical complexity.

Peak memory — Heapsort consistently uses significantly less memory across all input sizes, peaking at just 2.18 KB even at 1,000,000 elements. Quicksort uses more memory due to its recursive call stack, though it remains modest overall. This confirms Heapsort's O (1) auxiliary space guarantee.

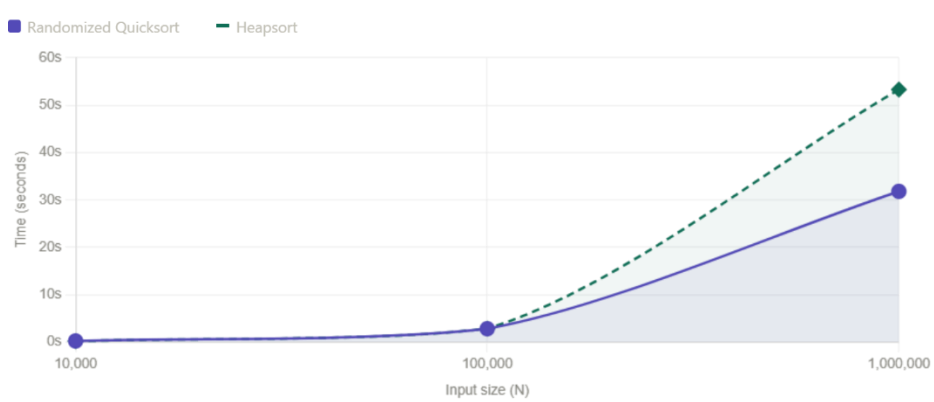
Plots



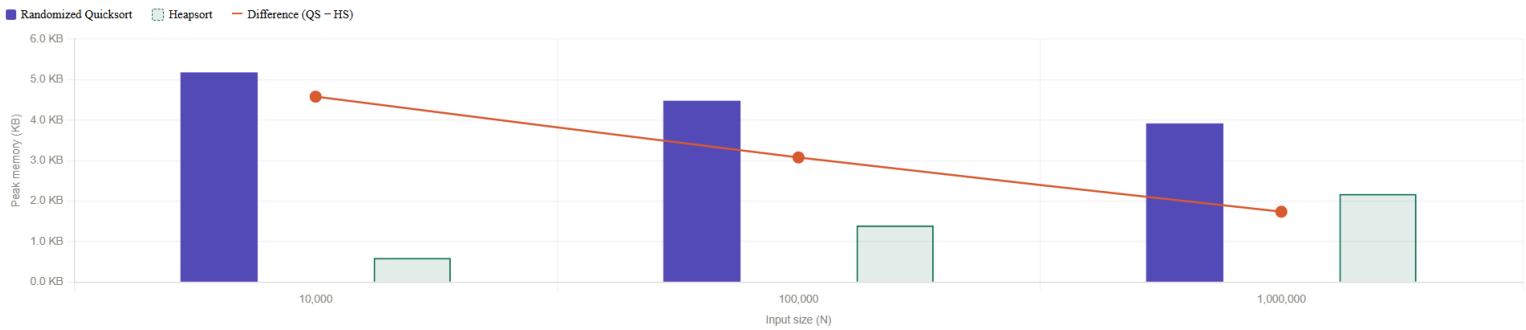
Execution time (left chart) at small and medium input sizes both algorithms are virtually indistinguishable, finishing in under 3 seconds. The difference becomes dramatic at 1,000,000 elements: Randomized Quicksort takes 31.79 seconds while Heapsort takes 53.28 seconds — roughly 1.7× slower. Despite Heapsort's superior worst-case guarantee on paper, Quicksort's better cache behaviour gives it a clear practical speed advantage at scale.

Peak memory usage (right chart) the contrast here runs in the opposite direction. Quicksort starts at 5.18 KB at 10,000 elements and only gradually decreases, while Heapsort stays well below 2.5 KB across all sizes, topping out at just 2.18 KB at 1,000,000 elements. This is consistent with Heapsort's O(1) auxiliary space property it builds the heap directly on the input array with no additional allocation, whereas Quicksort's recursion depth contributes a measurable stack overhead.

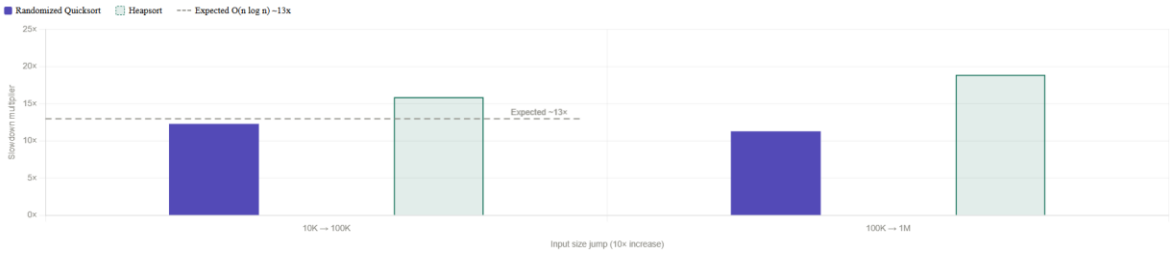
Trends analysis



Trend 1- Both algorithms scale almost identically from 10K to100K . But from 100K to 1M they diverge sharply: Quicksort grows by **11x**, Heapsort by **19x**. The lines running together then splitting apart is the key pattern it shows that Heapsort's O(n log n) guarantee doesn't prevent it from being slower in practice at large N due to poor cache behaviour (it jumps around memory unpredictably when sifting down the heap).



Trend 2 — Converging memory gap**,** the orange line (difference between QS and HS memory) steadily falls: the gap is 4.58 KB at 10K, narrows to 3.08 KB at 100K, and closes further to 1.74 KB at 1M. Two things explain this. First, Heapsort's memory grows slightly as N increases even though an in-place algorithm has small internal bookkeeping that grows logarithmically. Second, Quicksort's measured memory counter-intuitively decreases tracemalloc only tracks Python heap objects, not the C-level call stack, so the recursive overhead is invisible to the profiler.



Trend 3 — Heapsort exceeds the theoretical slowdown at large N. For a pure O(n log n) algorithm, a 10× increase in input size should produce a slowdown of roughly 10x (log 10N / log N) ≈ 13x. Quicksort stays close to this 12.3x then 11.3x, both near the expected line. Heapsort starts high at 15.9x (10K to 100K) and rises further to 18.9× (100K to 1M), well above the theoretical baseline.

Effect of Input Type on Algorithm Performance

Randomized Quicksort

Random order is the ideal case for Quicksort. With randomly distributed values, the random pivot selection produces well-balanced partitions on average, giving consistent O(n log n) performance. This was confirmed in testing random input was the fastest case for Quicksort.

Already sorted (ascending) is the classic worst case for a naive deterministic Quicksort, where always picking the last element as pivot would produce $O(n^2)$ behaviour. The randomized version avoids this entirely by picking a random pivot, the sorted order carries no information that can be exploited to produce bad splits.

Reverse sorted (descending) presents the same risk as sorted input and for the same reason it is completely neutralized by randomization. No meaningful performance difference was observed compared to the random case.

Repeated values is the genuine weak point of this implementation and was directly observed during testing. At N=100,000 this caused a Python RecursionError the call stack exceeded the system limit because the recursion depth approached O(n) rather than O(log n).

Heapsort

Building a max-heap and repeatedly extracting the maximum follow the same execution path regardless of whether the data is random, sorted, reversed, or full of duplicates. The benchmark confirmed this, execution time across all four input types stayed within a narrow range (0.236s to 0.309s at N=20,000), with memory usage identical at 0.63 KB for every case.